

Serverless Architectures TASK – 5

1.Design an event-driven workflow for a music-sharing app where users upload audio files and the app automatically extracts and stores the song's metadata (title, artist, duration) in a database.

Ans :

Use case: A user uploads an audio file to a music-sharing app.

1. The audio file is uploaded to an Amazon S3 bucket. S3 emits an ObjectCreated event that triggers an AWS Lambda function.
2. Lambda downloads/reads the audio metadata and extracts the title, artist, and duration.
3. Lambda stores the extracted metadata in Amazon DynamoDB, using the S3 object key or song ID as the primary key.
4. API Gateway can expose an endpoint for the app to retrieve the stored song metadata, while S3 handles the audio files themselves.

2.Implement a serverless function (such as AWS Lambda or Azure Function) that triggers when a new image is uploaded to a cloud storage bucket and writes the image's filename, upload time, and file size to a NoSQL database table.

Ans :

1. DynamoDB table

Create a table named ImageMetadata with:

- **Partition key:** filename
- **Type:** String

2. Lambda function

Configure the Lambda function to be triggered by an **S3 ObjectCreated** event.

```
import boto3

from datetime import datetime, timezone

from urllib.parse import unquote_plus


dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("ImageMetadata")


def lambda_handler(event, context):
    for record in event["Records"]:
        bucket = record["s3"]["bucket"]["name"]
        key = unquote_plus(record["s3"]["object"]["key"])
        file_size = record["s3"]["object"]["size"]

        upload_time = datetime.now(timezone.utc).isoformat()

        item = {
            "filename": key,
            "uploadTime": upload_time,
            "fileSize": file_size,
            "bucket": bucket
        }

        table.put_item(Item=item)
```

```
print(f'Saved metadata: {item}')
```

```
return {  
    "statusCode": 200,  
    "body": "Image metadata saved successfully"  
}
```

3. Configure the S3 trigger

In **Lambda** → **your function** → **Add trigger**:

1. Select **S3**.
2. Select your image bucket.
3. Set the event type to **All object create events**.
4. Optionally add a suffix filter such as .jpg or .png.
5. Confirm the trigger.

The workflow is:

User uploads image



Amazon S3

↓ ObjectCreated event

AWS Lambda



Extract filename, size, upload time



DynamoDB (ImageMetadata)

4. Required IAM permission

The Lambda execution role needs permission to write to DynamoDB, for example:

```
{
  "Effect": "Allow",
  "Action": [
    "dynamodb:PutItem"
  ],
  "Resource": "arn:aws:dynamodb:*:*:table/ImageMetadata"
}
```

Lambda also needs its standard **CloudWatch Logs** permissions so you can inspect the `print()` output.

5. Example DynamoDB item

After uploading `holiday.jpg`, the table could contain:

```
{
  "filename": "holiday.jpg",
  "uploadTime": "2026-10-03T12:45:30.123456+00:00",
  "fileSize": 245678
}
```

3. Build a notification system for a Zomato-style food delivery app: when a new menu PDF is uploaded by a restaurant, trigger a function that sends a confirmation email to the restaurant owner and logs the event in a database.
Hint: Use cloud storage event triggers and a transactional email API (like SendGrid or AWS SES) for the email step.

Ans :

1. Create the DynamoDB table

Create a table called MenuUploadEvents with:

- Partition key: eventId — String
- Attributes: restaurantId, filename, uploadTime, status

2. Lambda function

Configure an S3 ObjectCreated trigger for your menu bucket.

The Lambda can then send the email through SES and log the upload:

```
import boto3

import uuid

import os

from datetime import datetime, timezone

from urllib.parse import unquote_plus


s3 = boto3.client("s3")

ses = boto3.client("ses")

dynamodb = boto3.resource("dynamodb")


table = dynamodb.Table("MenuUploadEvents")


RESTAURANT_OWNER_EMAIL =
os.environ["RESTAURANT_OWNER_EMAIL"]


def lambda_handler(event, context):

    for record in event["Records"]:
```

```
bucket = record["s3"]["bucket"]["name"]
filename = unquote_plus(record["s3"]["object"]["key"])

upload_time = datetime.now(timezone.utc).isoformat()
event_id = str(uuid.uuid4())

# Log upload event
table.put_item(
    Item={
        "eventId": event_id,
        "restaurantId": "restaurant-001",
        "filename": filename,
        "uploadTime": upload_time,
        "status": "uploaded"
    }
)

# Send confirmation email
ses.send_email(
    Source=RESTAURANT_OWNER_EMAIL,
    Destination={
        "ToAddresses": [RESTAURANT_OWNER_EMAIL]
    },
    Message={
```

```

    "Subject": {
        "Data": "Menu uploaded successfully"
    },
    "Body": {
        "Text": {
            "Data": (
                f"Your menu file '{filename}' was uploaded "
                f"successfully at {upload_time}."
            )
        }
    }
}
)

```

```

print(f"Processed menu upload: {filename}")

```

```

return {
    "statusCode": 200,
    "body": "Menu upload processed successfully"
}

```

3. Configure permissions

The Lambda execution role needs permissions for:

- dynamodb:PutItem on MenuUploadEvents
- ses:SendEmail or ses:SendRawEmail

- CloudWatch Logs permissions

SES also requires the sender identity to be verified. If the account is still in the SES sandbox, the recipient generally needs to be verified as well.

4. Configure the S3 trigger

Set the S3 event to **ObjectCreated** and optionally add a suffix filter:

.pdf

Then uploading:

menus/restaurant-001/menu.pdf

4. Review a classmate's event-driven workflow diagram and provide 2 suggestions for improving scalability or error handling in their design.

Ans :

1. Improve scalability: Add Amazon SQS between S3 and Lambda. This buffers bursts of menu uploads and lets Lambda process messages asynchronously without overwhelming downstream services.
2. Improve error handling: Configure an SQS Dead-Letter Queue (DLQ) or Lambda failure destination. If sending the email or writing to DynamoDB repeatedly fails, the event can be captured for later investigation instead of being lost.